

Osmobitni Studentski Mikroprocesor i Assembler za Nastavu

OSMAN

Studenti: Bećarević Kemal, Đuderija Amina, Helać Hana

Profesor: Hrnjić Tarik

Sarajevo, 2026

Table of Contents

Uvod.....	3
ISA	3
Kodiranje i tipovi instrukcija	3
Spisak instrukcija	5
Asembler i pseudo-instrukcije	14
Organizacija RAM-a	12
Registarska datoteka	10
Pipeline i pregradni registar	12
Kontrolna jedinica.....	Error! Bookmark not defined.
Open questions.....	12
Detaljna dokumentacija svih instrukcija.....	14
NOP.....	14
ADD.....	14

Uvod

Ovaj rad dokumentuje dizajn i implementaciju 8-bitnog RISC mikroprocesora pod nazivom OSMAN. Procesor je baziran na Harvard arhitekturi i zamišljen je kao dvoadresna Load/Store mašina, sa 8-bitnom širinom podataka i adresnim prostorom. OSMAN posjeduje osam 8-bitnih registara, a također funkcioniše na bazi dvofazne protočne strukture. Opisani mikroprocesor će prvo biti implementiran u Logisim okruženju, nakon čega će biti hardverski realiziran (putem TTL komponenti ili FPGA razvojne ploče). U nastavku će biti detaljno opisan skup instrukcija, arhitektura procesora i sve ostale bitne karakteristike.

ISA

Kao što je rečeno, OSMAN predstavlja RISC procesor, odnosno procesor sa skraćenim skupom instrukcija. Konkretno, instrukcijski skup se sastoji od 31 instrukcije različitih tipova.

Kodiranje i tipovi instrukcija

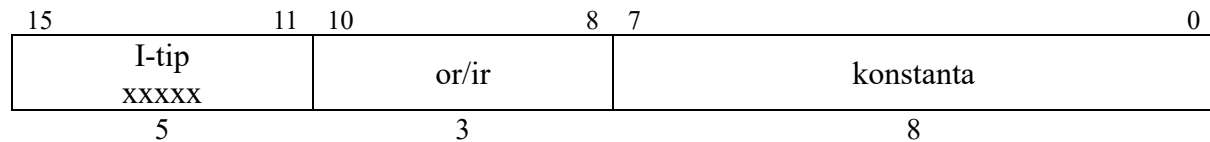
Sve OSMAN instrukcije su 16-bitne, ali se razlikuju po načinu kodiranja, ovisno od njihovog tipa. Prvih 5 bita svake instrukcije je rezervisano za njen opcode, dok se ostali biti koriste na različite načine ovisno od tipa instrukcije. Razlikujemo tri tipa OSMAN instrukcija:

R-tip: Instrukcije registarskog tipa, odnosno instrukcije koje rade isključivo sa dva registra. Sadrže dva operanda, od kojih je lijevi ujedno i odredište instrukcije. Način kodiranja je dat ispod.

15	11	10	8	7	5	4	0
R-tip xxxxx		or		ir		xxxxx	
5		3		3		5	

Kod instrukcija ovog tipa je donjih 5 bita neiskorišteno, jer će kontrolna jedinica na osnovu opcode-a generisati signal za ALU koji određuje operaciju, tako da nije potrebno više informacija osim opcode-a, odredišnog registra i izvornog registra. Polje 'or' predstavlja odredišni registar, ali ujedno i prvi operand instrukcije. Polje 'ir' je drugi izvorišni registar / operand instrukcije.

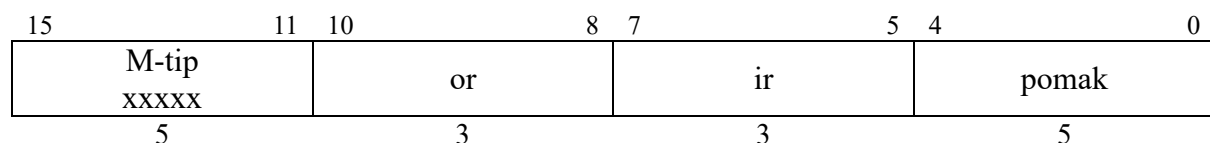
I-tip: Instrukcije koje koriste neposredne (immediate) vrijednosti. Ovaj tip pokriva veoma širok spektar instrukcija, od aritmetičkih instrukcija do skokova i grananja. Način kodiranja je predstavljen ispod.



Instrukcije I tipa rade samo sa jednim registrom, koji može biti ili izvorni ili odredišni, ovisno o instrukciji.

M-tip: Instrukcije indeksnog adresiranja. Ove instrukcije su slične instrukcijama R tipa, samo što se donjih 5 bita koristi za postavljanje pomaka pri adresiranju. Ovo znači da se pomoću ove vrste adresiranja može adresirati ± 16 adresa u odnosu na bazni registar, koji je zadan poljem *ir*. Polje *or* se može koristiti kao izvorni ili kao odredišni registar, ovisno od instrukcije.

Bitno je napomenuti da sa trenutnom arhitekturom i načinom kodiranja instrukcija R tipa, M tip u suštini i nije potreban. Donjih 5 bita ovih instrukcija koji se koriste za pomak se mogu bez problema kodirati u donjih 5 bita instrukcija R tipa, koji su trenutno neiskorišteni. Razlog za kreiranje M tipa je bio to što su instrukcije R tipa prvobitno bile kodirane na način da im je opcode jednak 0, a da se u donjih 5 bita kodira željena operacija. To je nakon proširivanja opcode-a sa 4 na 5 bita promijenjeno i preuzet je sistem gdje svaka instrukcija R tipa ima vlastiti opcode, u svrhu jednostavnije realizacije kontrolne jedinice. U svakom slučaju, M tip ćemo ostaviti zasebnim i neće se spajati sa R tipom, u slučaju da se nekad pojavi potreba za vraćanjem na stari sistem kodiranja instrukcija R tipa ili pronađe neka druga upotreba za njihovih donjih 5 bita.

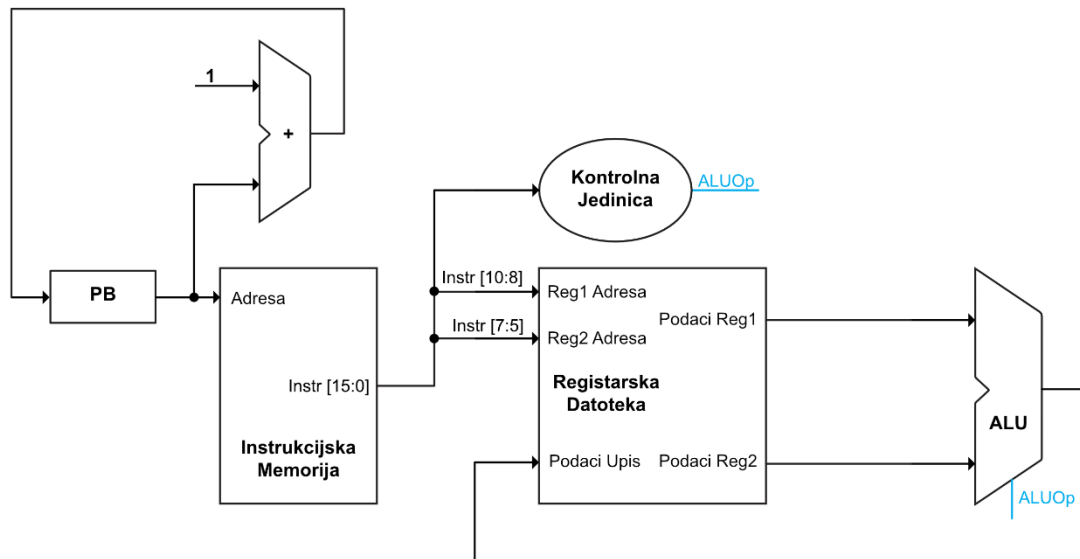


Spisak instrukcija

Instrukcija	Mnemonik	Opcode	Tip	Znacenje
Prazna instrukcija	NOP	00000	-	
Saberi	ADD	00001	R	
Oduzmi	SUB	00010	R	
Logicka negacija	NOT	00011	R	
Logicka konjukcija	AND	00100	R	
Logicka disjunkcija	OR	00101	R	
Ekskluzivna disjunk.	XOR	00110	R	
Pomjeri lijevo	SL	00111	R	
Pomjeri desno	SR	01000	R	
Saberi sa konstantom	ADDI	01001	I	
Oduzmi konstantu	SUBI	01010	I	
Log. konj. sa konst.	ANDI	01011	I	
Log. disj. sa konst.	ORI	01100	I	
Eks. disj. sa konst.	XORI	01101	I	
Poziv procedure	CALL	01110	I	
Povratak iz procedure	RET	01111	I	
Čitaj iz memorije sa konstantne adrese	LDI	10000	I	
Piši u memoriju na konstantnu adresu	STI	10001	I	
Učitaj konstantu	LOC	10010	I	
Bezuslovni skok	JMP	10011	I	
Skok ako je ZF=1	BEQ	10100	I	
Skok ako ZF=0	BNE	10101	I	
Skok ako je A<B	BLT	10110	I	
Skok ako je A≥B	BGE	10111	I	
Skok ako je CF=1	BC	11000	I	
Skok ako je CF=0	BNC	11001	I	
Indeksno čitanje iz memorije	LD	11010	M	
Indeksno pisanje u memoriju	ST	11011	M	
Učitaj efektivnu adresu	LEA	11100	M	
Uporedi 2 registra	CMP	11101	R	
Uporedi registar sa konstantom	CMPI	11110	I	

Realizacija arhitekture

Na početku će biti predstavljena jednostavna shema realizacije, koja odgovara arhitekturi koja je dovoljna kako bi se realizovale instrukcije R tipa, nakon čega ćemo postepeno dodavati ostale elemente i tipove instrukcija dok ne dobijemo finalnu shemu OSMAN-a.



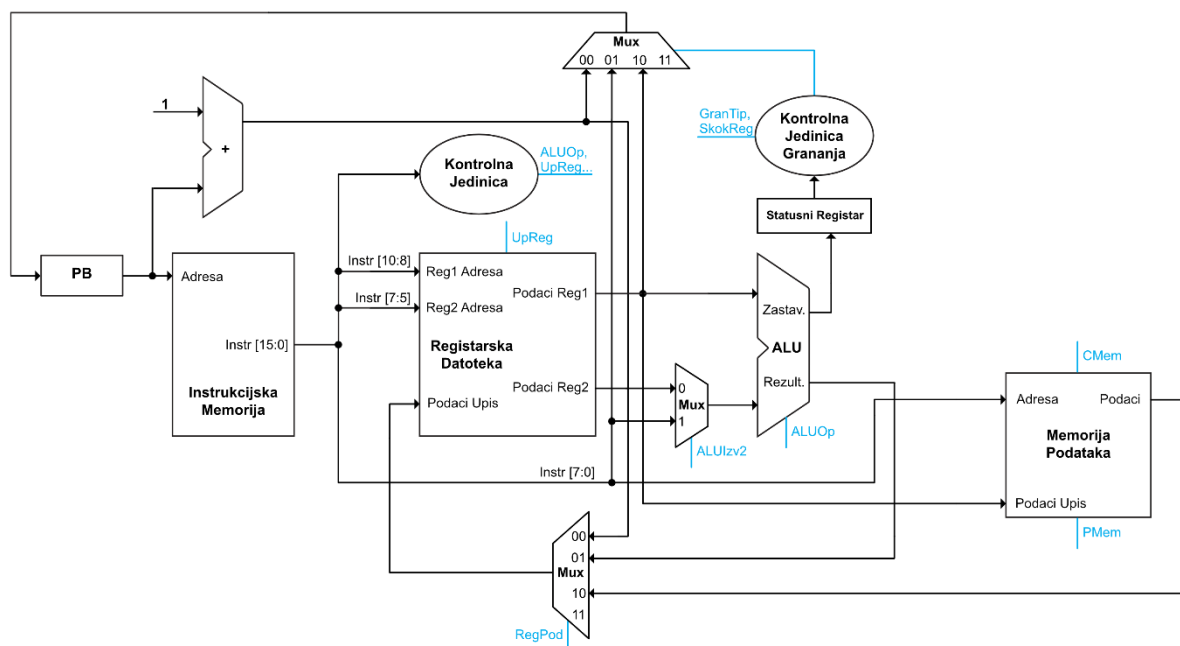
Slika 1: Arhitektura procesora za R-tip instrukcija

Kao što se može primjetiti, realizacija je veoma jednostavna. Kontrolna jedinica u ovom slučaju služi samo za generisanje ALUOp signala, koji se generiše na osnovu opcode-a instrukcije.

Registarska datoteka ima tri ulaza, pri čemu prva dva primaju adrese registara operanada *or* i *ir*. Treći ulaz je ulaz za podatak koji se upisuje u određeni registar, čija je adresa ista kao i za ulaz *Reg1 Adresa*. Generalno, ukoliko se kod bilo kakve instrukcije (bilo kojeg tipa) vrši upis u registar, adresa tog registra će biti jednaka polju *or*, odnosno bitima 10-8, što se može ožičiti interno unutar registarske datoteke. Registarska datoteka ima dva izlazna podatka, koji predstavljaju pročitane vrijednosti specificiranih registara. U slučaju instrukcija R tipa, ove se vrijednosti šalju direktno na ALU, čiji se izlaz opet direktno vraća na ulaz za upis u registarsku datoteku. Registarska datoteka će kasnije dobiti i *UpReg* kontrolni signal koji kontroliše upis, ali on ovdje nije potreban jer sve instrukcije R tipa vrše upis u registar.

Bitna napomena je da registarska datoteka treba biti implementirana tako da se čitanje vrši na uzlaznoj ivici sata, dok se pisanje treba vršiti na silaznoj ivici, kako ne bi došlo do “race condition” problema.

Situacija se značajno komplikuje dodavanjem podrške za I-tip instrukcija, najviše zbog širokog spektra instrukcija koje taj tip pokriva.



Slika 2: Arhitektura procesora sa podrškom za I i R tipove instrukcija

Krenut ćemo sa izmjenama na registarskoj datoteci. Prvo, dodan je kontrolni signal *UpReg* koji određuje da li se vrši upis podataka koji se nalaze na ulazu u registarsku datoteku. Također, prije ulaza *Podaci Upis* je dodan 4:1 multiplekser, koji bira između tri moguća podatka koja se mogu upisati u registar. Prvi je izlaz iz sabirača koji daje izlaz $PB+1$, kako bi se povratna adresa pri pozivu procedura korektno mogla spasiti u PA registar. Drugi je rezultat iz ALU-a, te se koristi za instrukcije R tipa, i neke instrukcije I tipa. Treći podatak predstavlja učitane podatke iz memorije, koji se koristi za instrukcije poput LD. Ovaj multiplekser kontroliše dvobitni kontrolni signal *RegPod*.

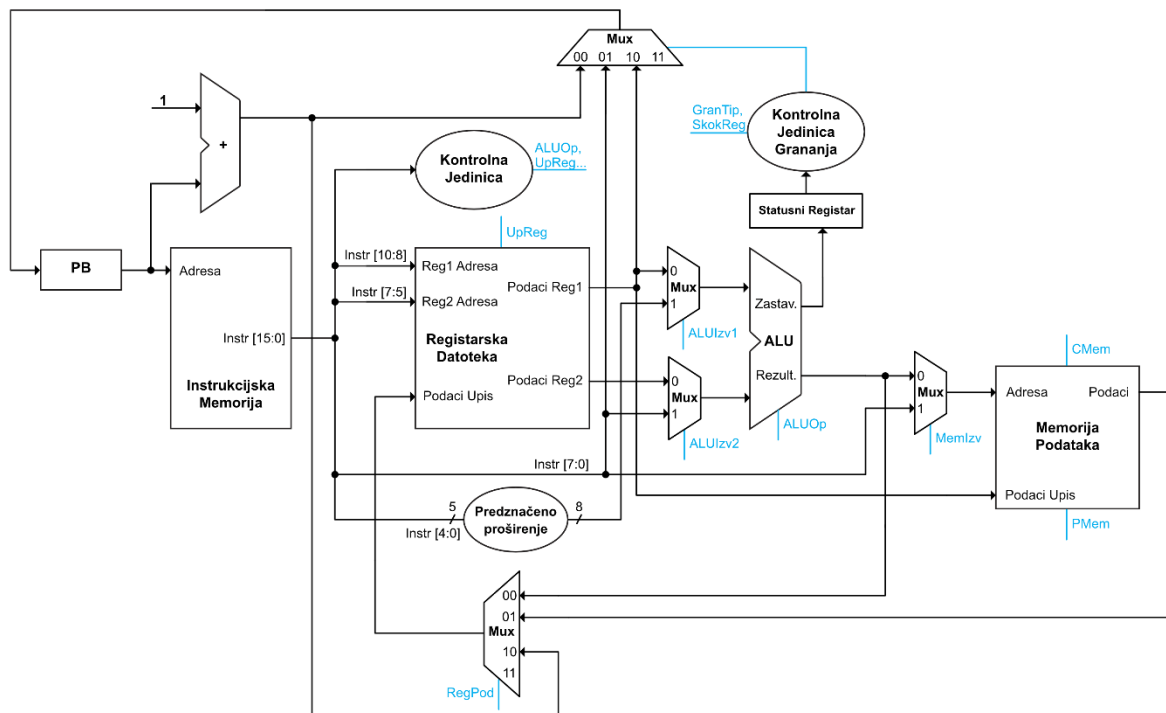
Ispred donjeg izlaza ALU-a je sada dodan 2:1 multiplekser, koji omogućava biranje između konstante specificirane u instrukcijama I tipa i vrijednosti registra zadanog *ir* poljem. Kontroliše ga kontrolni signal *AluIzv2*.

Arhitektura sada zahtijeva i memoriju podataka, koja je implementirana tako da ima dva kontrolna signala, *CMem* i *PMem*. Ovi signali kontrolišu da li se vrši čitanje ili pisanje iz memorije, respektivno. Memorija podataka ima dva ulaza, pri čemu je prvi tražena adresa, a drugi predstavlja ulaz za podatke koji se žele u nju upisati. Memorija posjeduje jedan izlaz, sa kojeg se mogu pročitati podaci sa adrese kada je to potrebno. Na ulaz za adresu se dovodi

donjih 8 bita instrukcije, jer je pomoću instrukcija I tipa memoriju moguće adresirati samo sa konstantnom adresom. Ulaz za podatke prima izlaz "Podaci Reg1" iz registarske datoteke, jer se podaci koji se žele upisati u memoriju nalaze u registru koji se kodira na bitima 10-8 instrukcije.

Realizacija instrukcija grananja zahtijeva dodavanje nekoliko komponenti. Prije svega, potrebno je implementirati statusni registar koji će pri instrukcijama poput CMP sačuvati vrijednosti zastavica koje ALU generiše. Ove se zastavice onda prosljeđuju kontrolnoj jedinici grananja, koja na osnovu kontrolnih signala *GranTip* i *SkokReg* generiše kontrolni signal za 4:1 multiplekser koji bira vrijednost koja se treba upisati u programski brojač. Prvi ulaz multipleksera je vrijednost PB+1. Druga vrijednost predstavlja donjih 8 bita instrukcije, te se koristi kod instrukcija poput JMP koje skaču na fiksnu adresu. Treći podatak je prvi izlaz iz registarske datoteke, koji je potreban kod RET instrukcije kako bi se skočilo na vrijednost spašenu u PA registru. Zbog toga je i uveden kontrolni signal *SkokReg*, koji kontrolna jedinica grananja koristi kako bi u tom slučaju postavila signal multipleksera na 10. Radi optimizacije kontrolne jedinice grananja bi bilo moguće dovesti prvi izlaz iz registarske datoteke i na ulaz 11 multipleksera ako bi to pojednostavilo logički izraz koji se dobije za kontrolnu jedinicu.

To su sve potrebne izmjene kako bi procesor podržao instrukcije R tipa i I tipa. Za implementaciju instrukcija M tipa je potrebno nešto manje izmjena, a šema procesora koji sada podržava sve tipove instrukcija je data ispod.



Slika 3: Arhitektura procesora koji podržava sve potrebne tipove instrukcija

Prije ulaza "Adresa" na memoriji podataka se sada nalazi 2:1 multiplexer, koji bira između konstante na bitima 7-0 i rezultata ALU operacije, te je potreban za realizaciju indeksnog adresiranja, gdje se adresa dobija sabiranjem baznog registra i pomaka. Ovaj multiplexer kontroliše kontrolni signal *MemIzv* i mora biti jednak logičkoj nuli kod instrukcija LD i ST koje koriste indeksno adresiranje, a jednak logičkoj jedinici kod instrukcija LDI i STI.

Druga izmjena je dodavanje komponente koja vrši predznačeno proširivanje polja *pomak* (biti 4-0) sa 5 bita na potrebnih 8 bita, kako bi se ta vrijednost mogla sabrati sa baznim registrom.

Treća izmjena je dodavanje multiplexera ispred gornjeg ulaza ALU-a. Bez ovog multiplexera ne bi bilo moguće sabrati vrijednosti *ir* i *pomak* polja kod instrukcija M tipa. Ovom izmjenom se omogućava da se bazni registar sabere sa proširenim *offset* poljem, pri čemu se *offset* šalje na gornji ulaz ALU-a, a polje *ir* na donji kao i sa svim instrukcijama koje koriste to polje, te da se taj rezultat konačno koristi kao adresa u memoriji podataka.

Ovim izmjenama su uspješno implementirane sve komponente koje su potrebne kako bi se realizirala svaka od 32 instrukcije definisane u ISA-i. U nastavku će biti detaljnije definisane neke od komponenti, poput kontrolnih jedinica i registarske datoteke.

Registarska datoteka

Kao što je rečeno ranije, OSMAN sadrži osam registara koji su vidljivi programeru. Također, postoji i statusni registar koji sadrži zastavice postavljene od strane određenih operacija i koristi se za realizaciju operacija grananja. Ovom registru nije moguće pristupiti. Od osam ponuđenih registara, 5 je opšte namjene, sa nazivima A-E. Registri SP i PA su namijenjeni za spremanje pokazivača na stek i povratne adrese iz procedura, respektivno. Na kraju ostaje registar PV, koji služi za čuvanje povratne vrijednosti pri radu sa procedurama. Bitno je naglasiti da je to stvar konvencije, te da je moguće taj registar, kao i registre SP i PA, koristiti i kao registre opšte namjene ukoliko je prijeko potrebno, uz veliku dozu opreza.

Kao što je spomenuto ranije, najbitnije za hardversku realizaciju registarske datoteke je da je implementirana na način da se upis vrši na silaznoj ivici, a čitanje na uzlaznoj, kako ne bi došlo do *race condition* problema.

Registar	Adresa	Namjena
SP	000	Pokazivač na stek
A	001	Registar opšte namjene
B	010	Registar opšte namjene
C	011	Registar opšte namjene
D	100	Registar opšte namjene
E	101	Registar opšte namjene
PV	110	Registar za spremanje povratne vrijednosti
PA	111	Registar za spremanje povratne adrese

Tabela 1: Dostupni registri i njihove namjene

Kontrolna jedinica

Kontrolna jedinica je odgovorna za generisanje kontrolnih signala na osnovu opcode-a instrukcije. Vrijednosti kontrolnih signala za sve instrukcije su date u tabeli 2.

Najpraktičniji način za hardversku realizaciju ove komponente bi bilo korištenje neke vrste ROM memorije, u koju bi se na adresu opcode-a neke instrukcije jednostavno upisali svi potrebni kontrolni signali. Iz tog razloga su u tabeli i kontrolni signali koji bi za neke instrukcije mogli biti nedefinisani zapisani kao '0'. U slučaju da se kontrolna jedinica ipak realizuje kao kombinatorni sklop, bilo bi korisno sve kontrolne signale koji su u nekom momentu nedefinisani označiti sa 'x', tj. „don't care“ u svrhu potencijalno jednostavnije realizacije sklopa.

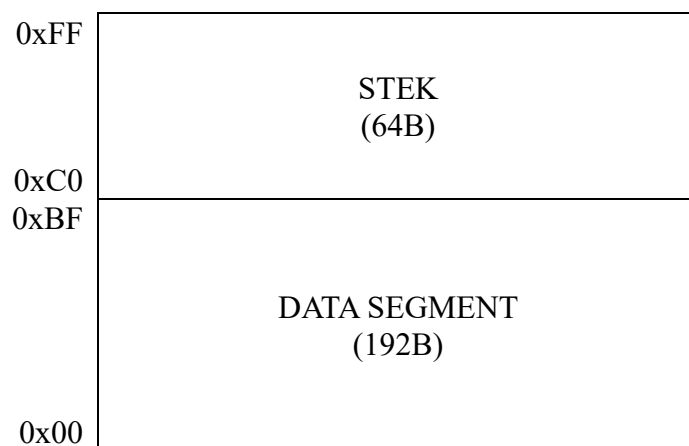
Instr	Tip	Gran Tip	SkokRe g	UpSt at	CMe m	PMe m	ALUO p	AluIzv 1	AluIzv 2	UpRe g	RegPo d	MemI zv
NOP	R	000	0	0	0	0	0000	0	0	0	00	0
ADD	R	000	0	0	0	0	0010	0	0	1	00	0
SUB	R	000	0	0	0	0	0011	0	0	1	00	0
NOT	R	000	0	0	0	0	0001	0	0	1	00	0
AND	R	000	0	0	0	0	0100	0	0	1	00	0
OR	R	000	0	0	0	0	0101	0	0	1	00	0
XOR	R	000	0	0	0	0	0110	0	0	1	00	0
SL	R	000	0	0	0	0	1000	0	0	1	00	0
SR	R	000	0	0	0	0	1001	0	0	1	00	0
ADDI	I	000	0	0	0	0	0010	0	1	1	00	0
SUBI	I	000	0	0	0	0	0011	0	1	1	00	0
ANDI	I	000	0	0	0	0	0100	0	1	1	00	0
ORI	I	000	0	0	0	0	0101	0	1	1	00	0
XORI	I	000	0	0	0	0	0110	0	1	1	00	0
CALL	I	111	0	0	0	0	0000	0	1	1	10	0
RET	I	000	1	0	0	0	0000	0	0	0	00	0
LDI	I	000	0	0	1	0	0000	0	1	1	01	1
STI	I	000	0	0	0	1	0000	0	1	0	01	1
LOC	I	000	0	0	0	0	0111	0	1	1	00	0
JMP	I	111	0	0	0	0	0000	0	1	0	00	0
BEQ	I	001	0	0	0	0	0000	0	1	0	00	0
BNE	I	010	0	0	0	0	0000	0	1	0	00	0
BLT	I	011	0	0	0	0	0000	0	1	0	00	0
BGE	I	100	0	0	0	0	0000	0	1	0	00	0
BCS	I	101	0	0	0	0	0000	0	1	0	00	0
BCC	I	110	0	0	0	0	0000	0	1	0	00	0
LD	M	000	0	0	1	0	0010	1	0	1	01	0
ST	M	000	0	0	0	1	0010	1	0	0	01	0
LEA	M	000	0	0	0	0	0010	1	0	1	00	0
CMP	R	000	0	1	0	0	0011	0	0	0	00	0
CMPI	I	000	0	1	0	0	0011	0	1	0	00	0

Tabela 2: Vrijednosti kontrolnih signala svih instrukcija

Kontrolna jedinica grananja

Organizacija RAM-a

Ispod je dat primjer organizacije RAM-a. Međutim, moguće je da će assembler biti napisan na način da sam određuje koliko je memorije potrebno za data segment, a da ostatak ostavlja steku. Također, moguće je da će biti potrebno odvojiti određeni broj adresa za memorijski mapirane „uređaje“, poput LED dioda radi interakcije sa vanjskim svijetom i sl.



Protočna struktura

U ovom dijelu će biti opisana realizacija protočne strukture, pregradnog registra, forwarding jedinice i prikazana nova shema arhitekture.

Jedinica za detekciju hazarda

Ukoliko se odlučimo da poslije instrukcija grananja nije obavezno stavljati NOP instrukciju, bit će potrebno uvesti jedinicu za detekciju hazarda kako bi se flushovale instrukcije u slučaju promašaja.

Open questions

Glavna pitanja:

- Da li moramo dodavati ono dijeljenje clocka sa delayevima jer tipa ako ram piše na uzlaznu jednostavno se neće stić izračunat vrijednost, a ako piše na silaznu onda se u memoriju neće stić upisat na silaznoj?????????????
- Da li imamo neki bitan benefit od toga sto nam se stek pomjera prema dole? Da li da napravimo da kreće od 0x00 prema gore pa da mozemo napraviti da reset pin samo uzme i resetuje registarsku datoteku sto ce postaviti SP na 0. U suprotnom, ili moramo dodavati logiku za setovanje registara na 0xFF, ili da assembler na pocetku svakog programa doda instrukciju koja sprema konstantu npr FF ili bilo koju drugu u SP. Posebno bitno zbog pozicioniranja I/O uredjaja.
- Da li se mora osigurati da se clock isto vrati na 1 ako se uradi reset? Jer ako resetujem u pola ciklusa moglo bi sve zeznut tj ako reset uradi dok je clock na 0 onda ce ta instrukcija nesto nedefinisano uraditi u tih pola ciklusa. Kako ovo implementirati?
- Najbolji pristup za IO uredjaje? Trenutno su LED memorijski mapirane a taster je prespojen direktno na pin. Da li sve moze preko registara pa da bude kao modularno ili stvari poput tastera i drugih ulaznih uredjaja moraju neki drugi tretman da imaju?
- U assembleru tehnicki dobijam vise tipova instrukcija, tipa svi skokovi se malo drugacije tretiraju jer im se polje za registar ne koristi. Ima li potrebe to razdvajati?
- Kakav nam je tretman predznacenih i nepredznacenih brojeva? Da li podrzavamo oboje. Vjv ne. Da li su nam postojece instrukcije skokova dovoljne za rad sa nepredznacnim brojevima. U svakom slucaju definisati ako podrzavamo samo predznacene da to dokumentujemo.

Todo:

- Skontati kako radi AS mode da se ne programira ploca svaki put.
- ROM je trenutno implementiran kao neki CASE u kodu. Trebalo bi u principu ici na ugradjenu memoriju. Kako ovo uraditi? Takodjer, da li se moze nekako to automatizovat jer sad svaki put kad uradim export on ce ponovo rom da prebrise i napravi kako hoce pa bi ja morao rucno. Ovo je generalno takodjer bitno da se moze dok radi ploca upisat neki program u ROM dinamicki.
- Testirati ovu metodu za pretvaranje hex brojeva (zbog negativnih) i generalni tretman brojeva negativnih u assembleru

Asembler i pseudo-instrukcije

Load address koji se prevodi u LI.

Smjesti lokalno I učitaj lokalno, INC DEC, INSP, DESP

Detaljna dokumentacija svih instrukcija

NOP

15	12	11	9	8	6	5	0
R-tip 0000		000		000		NOP 000000	
4		3		3		6	

Svrha: Instrukcija bez operacija

Format: NOP

Opis: NOP operacija se koristi kako bi se u tom ciklusu izvršavanja ne bi desilo ništa, tj. kako se stanje procesora i memorije ne bi mijenjalo instrukcijom.

ADD

15	12	11	9	8	6	5	0
R-tip 0000		or		ir		ADD 000001	
4		3		3		6	

Svrha: Sabiranje registara

Format: ADD or, ir

Značenje: $REG[or] \leftarrow REG[or] + REG[ir]$

Opis: Sabira vrijednosti registara *or* i *ir* i sprema rezultat u registar *or*.